



SIBDAS

Envio de Dados por GET

 **Atualização de Dados CLIENTES**

Nome Completo

Filipe Leandro Lima Gonçalves

Morada (NºPorta, Andar)

Praça Antunes Miranda n.º21 5º Drt

Código Postal

Cidade

Telefone

Email

Braga

982858385

filipe.855.leandro@outlook.com

Sexo

Data de nascimento

☒ Masculino ☐ Feminino

2003-11-23

Estado Civil

Sistema de Saúde

Profissão

Solteiro

CTT

✕ Cancelar

💾 Guardar

PHP e MySQL

- 1: Envio de Dados pelo Método GET
- 2: Encriptação de dados
- 3: Preencher o formulário com os dados do cliente
- 4: Atualizar os dados do cliente após submissão (POST)

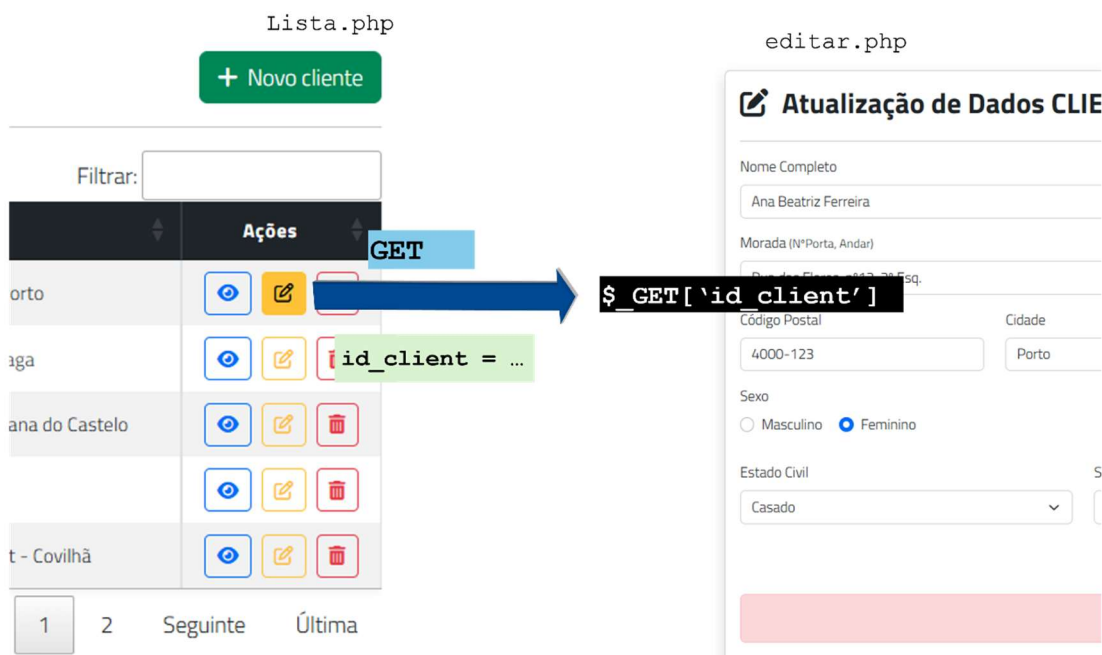
1 Envio de Dados pelo Método GET

Objetivo: Permitir que, ao clicar no botão  "Editar", o utilizador seja redirecionado para a página editar.php com o ID do cliente na URL, para que os dados desse cliente possam ser editados.

O Que Vamos Fazer

A funcionalidade que vamos implementar tem os seguintes objetivos:

1. Permitir que o utilizador edite os dados de um cliente já existente.
2. Fazer com que, ao clicar no botão  *Editar*, o sistema:
 - o Redirecione para `editar.php?id_client=...`
 - o Use esse `id_client` para obter os dados reais do cliente
 - o Preencha automaticamente o formulário
3. Após o utilizador modificar os dados e clicar em "Guardar":
 - o O formulário será enviado com POST
 - o Os dados serão **atualizados na base de dados**
 - o E o utilizador poderá ser **redirecionado para a lista com uma mensagem de sucesso**

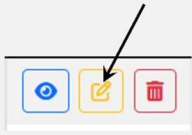


1. Identificar o local onde está a tabela de clientes

Abrir o ficheiro lista.php e localizar o `<td class="text-center">` com os botões de ação.

```
<td class="text-center">
  <a href="detalhes.php" class="btn btn-sm btn-outline-primary me-1">
    <i class="fa-solid fa-eye"></i>
  </a>
  <a href="editar.php" class="btn btn-sm btn-outline-warning me-1">
    <i class="fa-regular fa-pen-to-square"></i>
  </a>
  <a href="apagar.php" class="btn btn-sm btn-outline-danger">
    <i class="fa-solid fa-trash-can"></i>
  </a>
</td>
```

lista.php



Original

```
<a href="editar.php?id_cliente=<?= $cliente->id ?>" class="btn btn-sm btn-outline-warning me-1">
  <i class="fa-regular fa-pen-to-square"></i>
</a>
```

2. Substituir o botão "Editar" estático por um com link dinâmico

Atualizar o botão "Editar" para que o link inclua o ID do cliente.

Código original:

```
<a href="editar.php" class="btn btn-sm btn-outline-warning me-1">
  <i class="fa-regular fa-pen-to-square"></i>
</a>
```

Substituir por:

```
<a href="editar.php?id_cliente=<?= ($cliente->id ?>" class="btn btn-sm btn-outline-warning me-1">
  <i class="fa-regular fa-pen-to-square"></i>
</a>
```

Explicação

- `href="editar.php?id_cliente=<?= $cliente->id ?>"`
 - este link inclui o ID do cliente na URL.
- O `<?= $cliente->id ?>` é uma forma curta de escrever `<?php echo $cliente->id; ?>`, e imprime o valor do ID do cliente.
- Quando se clica no botão, a página `editar.php` recebe um parâmetro `id_cliente`, como por exemplo:
`editar.php?id_cliente=5`

3. Testar no navegador

- Abrir o ficheiro lista.php no browser.
- Clicar no botão  "Editar" de um cliente qualquer
- Verificar se a URL muda para algo como:

 http://127.0.0.1/isep-ginasio/private/views/clientes/editar.php?id_cliente=2

(Por enquanto) a página editar.php ainda pode estar estática, mas **já está a receber o ID na URL**.

Objetivo:

- Proteger o acesso à página editar.php, garantindo que só é acessível com sessão ativa.
- Permitir apenas pedidos do tipo GET.
- Obter e validar o parâmetro id_cliente enviado via URL.
- Impedir acesso não autorizado ou manipulação de dados via POST.

1. Proteger o acesso com sessão ativa. No topo do ficheiro editar.php, verificar se já existe de fichas anteriores o seguinte código incluir o seguinte código

```
require_once __DIR__ . '/../../includes/funcoes.php';  
redirect_if_not_logged();
```

Explicação:

- Verifica se a sessão está ativa.
- Se o utilizador não estiver autenticado, é redirecionado para a página de login.

2. Permitir apenas pedidos GET e POST Após garantir que a sessão está ativa, limitar os métodos permitidos a GET e POST:

```
if (!in_array($_SERVER['REQUEST_METHOD'], ['GET', 'POST'])) {  
    header('Location: ' . BASE_URL . '/public/login.php');  
    exit;  
}
```

Explicação:

- Permite o acesso à página tanto por GET (quando se clica no botão “Editar”) como por POST (quando o utilizador submete o formulário de atualização).
- Rejeita métodos indevidos como PUT, DELETE, OPTIONS, entre outros, reforçando a segurança da aplicação.

3. **Obter e validar o ID do cliente via GET.** De seguida, recolher o valor do cliente enviado na URL:

```
// Recolhe o ID do cliente da URL
$idClient = $_GET['id_cliente'] ?? null;

if (!$idClient) {
    header('Location: ' . BASE_URL . '/private/views/clientes/lista.php');
    exit;
}
```

Explicação:

- Garante que o parâmetro `id_cliente` está presente e é numérico.
- Caso contrário, o utilizador é redirecionado para a lista de clientes.

4. Incluir cabeçalho e navegação. Para manter o layout consistente com o restante do sistema, incluir os seguintes

```
include '../includes/header.php';
include '../includes/nav.php';
```

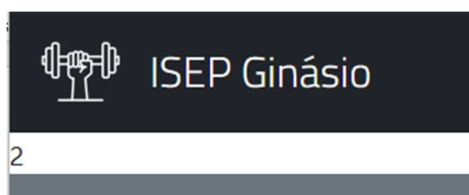
Para confirmar que o valor foi recebido corretamente, podes testar com:

```
// Para testar (temporário)
echo ($idClient);
```

Resultado Esperado

- O acesso a `editar.php` só é permitido a utilizadores com sessão iniciada.
- A página só é acedida via GET, a partir do botão “Editar”.
- Se o `id_cliente` for inválido ou estiver ausente, o utilizador é redirecionado.

 http://127.0.0.1/isep-ginasio/private/views/clientes/editar.php?id_cliente=2



Problema: Acesso Indevido a Dados de Outros Clientes

Imagine o seguinte cenário: Um utilizador com sessão iniciada acede à página `editar.php?id_cliente=2` e visualiza corretamente os dados do cliente nº 2. No entanto, se esse mesmo utilizador alterar manualmente o valor do ID na URL para `editar.php?id_cliente=6`, conseguirá ver e até editar os dados de outro cliente — sem qualquer validação adicional.

Como Reproduzir o Problema

1. Iniciar sessão como um utilizador comum
2. Aceder a:
`editar.php?id_cliente=2`
(visualiza os dados do cliente nº 2 — até aqui tudo certo).
3. Alterar manualmente a URL para:
`editar.php?id_cliente=6`
(consegue ver os dados do cliente nº 6, mesmo que não tenha permissão).



Observar que se consegue aceder aos dados de outro cliente — **isto é uma falha de segurança**. Não basta **proteger o acesso com sessão**, é também necessário garantir que o **utilizador autenticado só vê os dados que tem autorização para ver**.

2 Encriptação de dados

Objetivo: Encriptação e Desencriptação do ID do Cliente com OpenSSL (AES-256-CBC)

Evitar que o ID do cliente fique visível e facilmente manipulável na URL como:

`editar.php?id_cliente=6`

E passar a usar algo como:

`editar.php?id_cliente=7eaf0b16f0a5ab6d...`

Assim, protegemos a informação e prevenimos ataques por manipulação de URL.

1 – Definir chaves OpenSSL no `config.php`

2. Após as definições globais (APP_NAME, MYSQL_HOST, etc.), adicionar o seguinte bloco:

```
// -----  
// Segurança - Encriptação com OpenSSL  
// -----  
  
define('OPENSSL_METHOD', 'AES-256-CBC'); // Algoritmo simétrico robusto  
define('OPENSSL_KEY', 'H0SDRQzIGqc1X2kbYBk9xspdn9U5f3Wa'); // Chave de 32 caracteres  
define('OPENSSL_IV', 'BzKABjuREsHgnw56'); // Vetor de inicialização (16  
caracteres)
```

Notas :

- OPENSSL_METHOD define o algoritmo (AES-256-CBC é seguro e comum para dados simples).
- OPENSSL_KEY é uma chave privada (deve ter exatamente 32 caracteres para AES-256).
- OPENSSL_IV é um vetor de inicialização (com 16 caracteres).

2 – Criar as funções de encriptação e desencriptação

1. Abrir o ficheiro `private/includes/funcoes.php`.

2. Adicionar o seguinte código:

```
// =====  
// Encriptação e desencriptação de valores com OpenSSL  
// =====  
  
function aes_encrypt($value) {  
    return bin2hex(openssl_encrypt(  
        $value,  
        OPENSSL_METHOD,  
        OPENSSL_KEY,  
        OPENSSL_RAW_DATA,  
        OPENSSL_IV  
    ));  
}
```

```
function aes_decrypt($value) {
    if (!is_string($value) || strlen($value) % 2 != 0) return false; // proteção básica

    return openssl_decrypt(
        hex2bin($value),
        OPENSSL_METHOD,
        OPENSSL_KEY,
        OPENSSL_RAW_DATA,
        OPENSSL_IV
    );
}
```

Função	Função Técnica	O que faz
aes_encrypt()	Encriptação	Transforma um valor (ex: 6) em hexadecimal encriptado
aes_decrypt()	Desencriptação	Converte o hexadecimal encriptado de volta ao valor original
% 2 != 0	Validação mínima de segurança	Um hexadecimal válido tem número par de caracteres

Objetivo:

- Atualizar lista.php para usar aes_encrypt() nos links de edição.
- Atualizar editar.php para usar aes_decrypt() ao ler \$_GET['id_cliente'].
- Testar e validar funcionamento + tentativa de manipulação.

1. Usar aes_encrypt() em lista.php

- Abrir o ficheiro **lista.php** onde é gerado o botão “Editar”.
- Substituir o link com ID visível por um link com ID encriptado.

```
<a href="editar.php?id_cliente=<?=$cliente->id ?>" . . .
```

por:

```
<a href="editar.php?id_cliente=<?=$cliente->id ?>" . . .
```

Nota: O ID visível será substituído por uma string longa e ilegível como 7eaf0b16f0a5ab6d...

4 – Usar `aes_decrypt()` em `editar.php`. No ficheiro `editar.php`, após verificar que a sessão está ativa e que o pedido foi feito por GET ou POST, descriptar o valor do `id_cliente` recebido via URL.

Posicionamento correto: este bloco deve ser inserido imediatamente depois de:

```
require_once __DIR__ . '/../../includes/funcoes.php';
redirect_if_not_logged();

if (!in_array($_SERVER['REQUEST_METHOD'], ['GET', 'POST'])) {
    header('Location: ' . BASE_URL . '/public/login.php');
    exit;
}
```

Código para descriptação:

```
$idClientEncrypted = $_GET['id_cliente'] ?? null;
$idClient = aes_decrypt($idClientEncrypted);

if (!$idClient || !is_numeric($idClient)) {
    header('Location: ' . BASE_URL . '/private/views/clientes/lista.php');
    exit;
}
```

O que este bloco faz:

- Recolhe o valor encriptado da URL (`$_GET['id_cliente']`).
- Usa `aes_decrypt()` para recuperar o valor original.
- Garante que o valor descriptado é numérico.
- Se o valor não for válido, redireciona de volta para a lista de clientes.

5 – Testar o comportamento. Passos para testar:

- Aceder à lista de clientes.
- Clicar em “Editar” num dos clientes.
- Confirmar que a URL gerada contém um valor encriptado, ex.:
`editar.php?id_cliente=7eaf0b16f0a5ab6d...`
- A página `editar.php` deve carregar corretamente o cliente.
- Tentar alterar manualmente o valor encriptado → a página deve redirecionar (falha na descriptação).

Objetivo: Ligar-se à base de dados, procurar o cliente com o ID descriptado e, se encontrado, guardar os seus dados num objeto PHP para que possam ser usados no preenchimento automático do formulário de edição.

1. Obter os dados do cliente na base de dados. Depois de descriptar e validar o ID do cliente, é necessário **procurar o cliente na base de dados** e, se existir, **carregar os seus dados para preencher o formulário**.

Código a adicionar logo após a verificação do \$idCliente:

```
try {
    $ligacao = new PDO(
        "mysql:host=" . MYSQL_HOST . ";dbname=" . MYSQL_DATABASE . ";charset=utf8",
        MYSQL_USERNAME,
        MYSQL_PASSWORD
    );

    $ligacao->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    // Preparar e executar a query com segurança
    $stmt = $ligacao->prepare("SELECT * FROM clientes WHERE id = :id");
    $stmt->bindParam(':id', $idCliente, PDO::PARAM_INT);
    $stmt->execute();

    $cliente = $stmt->fetch(PDO::FETCH_OBJ);

    // Se não encontrou o cliente, redireciona
    if (!$cliente) {
        header('Location: ' . BASE_URL . '/private/views/clientes/lista.php');
        exit;
    }

    // $erro = ''; apagar senao o erro nao e exibido
} catch (PDOException $err) {
    $erro = "Erro na ligação à base de dados.";
    $cliente = null;
}

// Fecha a ligação
$ligacao = null;
```

3 Preencher o formulário com os dados do cliente

Objetivo: Substituir os valores escritos manualmente (valores fixos) no formulário HTML pelos dados reais do cliente que foram obtidos da base de dados. O formulário deve ser preenchido automaticamente com os dados do cliente, permitindo ao utilizador editar de forma segura e prática.

Instruções

Para cada campo do formulário, deves **substituir o valor estático** (escrito à mão) pelo valor proveniente da base de dados, utilizando o objeto `$cliente`. A função `htmlspecialchars()` é usada para evitar erros de HTML e prevenir ataques XSS.

1. Substituições Campo a Campo

Nome:

```
<input type="text" class="form-control" id="texto_nome" name="nome"
value="<?= htmlspecialchars($cliente->nome) ?>" required>
```

Morada:

```
<input type="text" class="form-control" id="texto_endereco" name="morada_cliente"
value="<?= htmlspecialchars($cliente->morada) ?>">
```

Cidade:

```
<input type="text" class="form-control" id="texto_cidade" name="cid_cliente"
value="<?= htmlspecialchars($cliente->cidade) ?>" required>
```

Telefone:

```
<input type="text" class="form-control" id="texto_cliente" name="tel_cliente"
value="<?= htmlspecialchars($cliente->telefone) ?>" required>
```

Email:

```
<input type="email" class="form-control" id="texto_email" name="email_cliente"
value="<?= htmlspecialchars($cliente->email) ?>" required>
```

Data de nascimento:

```
<input type="text" class="form-control" id="texto_dnasc" name="dnasc_cliente"  
value="<?= date('Y-m-d', strtotime($cliente->data_nascimento)) ?>" required>
```

Sexo (radio buttons):

```
<div class="form-check form-check-inline">  
  <input class="form-check-input" type="radio" name="radio_gender" id="radio_m"  
  value="m" <?= $cliente->sexo == 'm' ? 'checked' : '' ?>>  
  <label class="form-check-label" for="radio_m">Masculino</label>  
</div>  
<div class="form-check form-check-inline">  
  <input class="form-check-input" type="radio" name="radio_gender" id="radio_f"  
  value="f" <?= $cliente->sexo == 'f' ? 'checked' : '' ?>>  
  <label class="form-check-label" for="radio_f">Feminino</label>  
</div>
```

Sistema de Saúde:

```
<input type="text" class="form-control" id="texto_SSaude" name="campo_opcao"  
value="<?= htmlspecialchars($cliente->sistema_saude) ?>" list="sistemasaude">
```

Teste 1 – Confirmar carregamento dos dados

No formulário editar.php, confirmar que os campos estão preenchidos com os dados do cliente

Teste 2 – Validar dados com outro cliente

- Volta à lista.php e edita um cliente diferente.
- Garante que os dados no formulário mudam conforme o cliente.

Nota Importante Os campos **Código Postal**, **Profissão** e **Estado Civil** ainda não existem na base de dados, por isso **não são carregados automaticamente** no formulário a partir do objeto \$cliente

Vamos adicionar esses campos à base de dados mais à frente, através de um ALTER TABLE. Assim que forem criados, o preenchimento será feito da mesma forma que os restantes campos.

4 Atualizar os dados do cliente após submissão (POST)

Objetivo: Após o utilizador editar os dados no formulário, o sistema deve:

1. Detetar se o formulário foi submetido (via POST);
2. Validar os dados recebidos;
3. Atualizar os dados do cliente na base de dados;
4. Redirecionar para a lista com uma mensagem de sucesso (ou mostrar erro).

1. Atualizar o <form> para manter o ID encriptado na URL No início do formulário, altera o action="#" para incluir o ID encriptado:

```
<form action="editar.php?id_cliente=<?= $idClientEncrypted ?>" method="post" novalidate>
```

Assim, quando o formulário for submetido, o ID encriptado continua disponível na URL para que se possa usá-lo no processamento.

2 – Desencriptar e validar o ID do cliente Logo no início do ficheiro editar.php verificar se temos este código:

```
$idClientEncrypted = $_GET['id_cliente'] ?? null;  
$idClient = aes_decrypt($idClientEncrypted);  
  
if (!$idClient || !is_numeric($idClient)) {  
    header('Location: ' . BASE_URL . '/private/views/clientes/lista.php');  
    exit;  
}
```

3 – Detetar submissão via POST Logo após a verificação do ID

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {  
    $novoNome = $_POST['nome'] ?? '';  
  
    if (empty(trim($novoNome))) {  
        $erro = "O nome não pode estar vazio.";  
    } else {  
        try {  
            $ligacao = new PDO(  
                "mysql:host=" . MYSQL_HOST . ";dbname=" . MYSQL_DATABASE . ";charset=utf8",  
                MYSQL_USERNAME,  
                MYSQL_PASSWORD  
            );  
            $ligacao->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);  
  
            $stmt = $ligacao->prepare("UPDATE clientes SET nome = :nome WHERE id = :id");  
            $stmt->bindParam(':nome', $novoNome, PDO::PARAM_STR);  
            $stmt->bindParam(':id', $idClient, PDO::PARAM_INT);  
            $stmt->execute();  
  
            // Mensagem de sucesso e redirecionamento (opcional)  
            header('Location: lista.php');  
            exit;  
        } catch (PDOException $err) {  
            $erro = "Erro ao atualizar o nome: " . $err->getMessage();  
        }  
    }  
}
```

Motivo para esta ordem. Porquê colocar este código antes do SELECT que carrega os dados do cliente?

- O ID já está descriptado
- O POST é tratado primeiro
- Evita fazer SELECT duas vezes
- Fluxo claro: se for POST → atualizar; se for GET → mostrar

4. Depois disto, testar:

- Alterar o nome no formulário.
- Clicar em “Guardar”.
- Deve ser redirecionado para lista.php.
- Verificar se o nome foi mesmo alterado na base de dados e reaparece na lista.php.

5. Alterar o código para que possa ser feito o update do nome, email, morada e telefone

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {  
    $novoNome = $_POST['nome'] ?? '';  
    $novoEmail = $_POST['email_cliente'] ?? '';  
    $novaMorada = $_POST['morada_cliente'] ?? '';  
    $novoTelefone = $_POST['tel_cliente'] ?? '';  
  
    if (empty(trim($novoNome))) {  
        $erro = "O nome não pode estar vazio.";  
    } else {  
        try {  
            $ligacao = new PDO(  
                "mysql:host=" . MYSQL_HOST . ";dbname=" . MYSQL_DATABASE . ";charset=utf8",  
                MYSQL_USERNAME,  
                MYSQL_PASSWORD  
            );  
            $ligacao->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);  
  
            # $stmt = $ligacao->prepare("UPDATE clientes SET nome = :nome WHERE id = :id");  
            $stmt = $ligacao->prepare("  
                UPDATE clientes  
                SET nome = :nome,  
                    email = :email,  
                    morada = :morada,  
                    telefone = :telefone  
                WHERE id = :id  
            ");  
  
            # $stmt->bindParam(':nome', $novoNome);  
            $stmt->bindParam(':nome', $novoNome, PDO::PARAM_STR);  
            $stmt->bindParam(':email', $novoEmail, PDO::PARAM_STR);  
            $stmt->bindParam(':morada', $novaMorada, PDO::PARAM_STR);  
            $stmt->bindParam(':telefone', $novoTelefone, PDO::PARAM_STR);  
            $stmt->bindParam(':id', $idClient, PDO::PARAM_INT); // ou $  
            $stmt->execute();  
  
            // Mensagem de sucesso e redirecionamento (opcional)  
            header('Location: lista.php');  
            exit;  
  
        } catch (PDOException $err) {  
            $erro = "Erro ao atualizar o nome: " . $err->getMessage();  
        }  
    }  
}
```

6. Instruções de teste

1. Preencher e submeter o formulário com novos dados nos campos:

- Nome
- Email
- Morada
- Telefone

2. Confirmar que:

- Os dados são guardados corretamente na base de dados.
- A página redireciona para lista.php sem erro.
- Os novos dados aparecem na listagem.

7. Verificar tratamento de erros. Se o campo “Nome” for enviado em branco, o código está preparado para apresentar uma mensagem de erro no topo do formulário. Esse comportamento é útil para validar obrigatoriedades antes de tentar atualizar a base de dados. Verificar se existe o seguinte bloco de validação de erro. Se não existir, deve ser adicionado:

```
<?php if (!empty($erro)): ?>
    <div class="alert alert-danger text-center" role="alert">
        <?= htmlspecialchars($erro) ?>
    </div>
<?php endif; ?>
```

8 Teste:

- Apagar o conteúdo do campo “Nome”.
- Clicar em “Guardar”.
- Confirmar que aparece a mensagem:

O nome não pode estar vazio.

Objetivo: Evitar duplicação de código ao validar o nome do cliente, criando uma função genérica que pode ser utilizada em vários ficheiros (por exemplo: novo.php e editar.php)

1 – Criar ficheiro validacoes.php

- No diretório includes, cria um novo ficheiro chamado **validacoes.php**.
- Dentro do ficheiro, define a função para validar o nome:

```
<?php

function validar_nome(string $nome): array {
    $erros = [];

    if (empty(trim($nome))) {
        $erros[] = "O campo Nome é obrigatório.";
    } elseif (preg_match('/\d/', $nome)) {
        $erros[] = "O campo Nome não pode conter números.";
    }

    return $erros;
}
```

2 – Incluir o ficheiro onde for necessário No início de novo.php e editar.php, logo após incluir funcoes.php, adicionar:

```
require_once __DIR__ . '/../includes/funcoes.php';
redirect_if_not_logged();
require_once __DIR__ . '/../includes/validacoes.php';
```

3. Usar a função validar_nome no tratamento de formulário. Exemplo em editar.php:

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $novoNome = $_POST['nome'] ?? '';

    $erros = validar_nome($novoNome);

    if (empty($erros)) {
        // Atualizar base de dados...
        try {
            ...
        }
    }
}
```


4. Mostrar os erros no formulário

Temos isto corretamente implementado:

```
<?php if (!empty($erros)): ?>
    <div class="alert alert-danger text-center" role="alert">
        <?= htmlspecialchars($erros) ?>
    </div>
<?php endif; ?>
```

Mas como estamos agora a usar \$erros (array), devemos adaptar para mostrar todos. Substituir por:

```
<?php if (!empty($erros)): ?>
    <div class="alert alert-danger text-center" role="alert">
        <?php foreach ($erros as $erro): ?>
            <div><?= htmlspecialchars($erro) ?></div>
        <?php endforeach; ?>
    </div>
<?php endif; ?>
```

5. Teste

1. Submeter o formulário com o nome vazio → deve aparecer: "O campo Nome é obrigatório."
2. Submeter com números no nome → deve aparecer: "O campo Nome não pode conter números."
3. Submeter com nome válido → não aparecem erros.

Objetivo: Evitar duplicação de código e melhorar a organização, reutilizando a função `validar_nome()` definida em `validacoes.php`, em vez de escrever a validação diretamente no ficheiro.

1. Comentar a validação antiga. No ficheiro `novo.php`, localiza a parte do código onde estavas a validar o nome diretamente:

```
// Verificar se o campo está vazio
/*
if (empty($nome)) {
    $erros[] = "O campo Nome é obrigatório.";
}
// Verificar se contém apenas números ou mistura de letras com números
elseif (preg_match('/\d/', $nome)) {
    $erros[] = "O campo Nome não pode conter números.";
}
*/
```

Deixar este código comentado como referência para estudo e comparação.

2. **Substituir pela função `validar_nome()`.** Logo a seguir, adicionar a chamada à função reutilizável:

```
$erros = validar_nome($nome);
```

Esta função vai devolver um array com os erros encontrados (ou um array vazio se não houver erros).

3 – Validar antes de inserir Certificar de que só tentamos inserir os dados se não houver erros:

```
if (empty($erros)) {  
    // Inserir novo cliente...  
}
```

Ficheiro novo.php

```
if ($_SERVER["REQUEST_METHOD"] == "POST") {  
    $nome = $_POST["nome_cliente"] ?? '';  
  
    $erros = validar_nome($nome);  
  
    if (empty($erros)) {  
        // Inserir novo cliente...  
    }  
}
```

Resultado Esperado

- O código fica mais limpo e reutilizável.
- A validação do nome está centralizada num único local (`validacoes.php`).
- Facilita a manutenção e expansão (ex: no futuro podes validar telefone, email, etc., da mesma forma).

4. Aplicar a mesma abordagem usada na validação do nome a todos os outros campos do formulário, criando **funções específicas** no ficheiro `validacoes.php`.